

Backtracking Fundamentals

1 Introduction

Definition

Backtracking is a systematic search technique that builds a solution one decision at a time, checks whether the partial solution can still become valid, and abandons a branch as soon as it becomes impossible.

Backtracking is refined brute force: it checks partial candidates while they are being built instead of waiting until a full candidate is complete.

Core Pattern

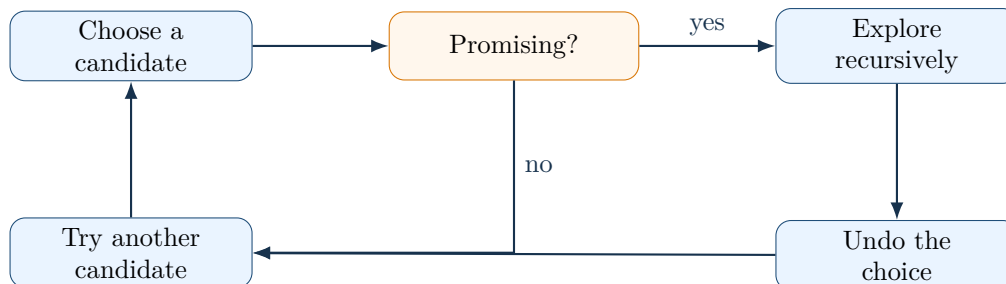
The essential cycle is:

Choose → Explore → Undo → Try another choice

Here the focus is on feasible-solution search, not optimisation.

2 Backtracking Control Flow

At each recursive level, the algorithm chooses a candidate, tests it, explores it if valid, and then undoes the choice.



Key Point

The undo step restores the previous partial solution before the next candidate is tried.

3 Brute Force vs Backtracking

Aspect	Brute Force	Backtracking
Checking	After complete candidates.	During construction.
Search space	Often explores everything.	Prunes bad branches early.
Traversal	Exhaustive generation.	Usually depth-first search.

Main Contrast

Brute force checks full candidates. Backtracking rejects impossible partial candidates.

4 State-Space Tree

State-Space Tree

A **state-space tree** represents all possible decisions of a search problem.

- The root represents the empty solution.
- Each internal node represents a partial solution.
- Each edge represents a choice.
- Each leaf is a solution or a dead end.

Backtracking walks this tree depth first and cuts off nonpromising subtrees.

Promising Test

Let S_k be the partial solution after k decisions.

$$P(S_k) = \begin{cases} \text{true}, & \text{if } S_k \text{ may still become valid,} \\ \text{false}, & \text{if } S_k \text{ cannot become valid.} \end{cases}$$

Backtracking explores S_k only when

$$P(S_k) = \text{true.}$$

5 Worked Arrangement Example

Problem

Arrange three people

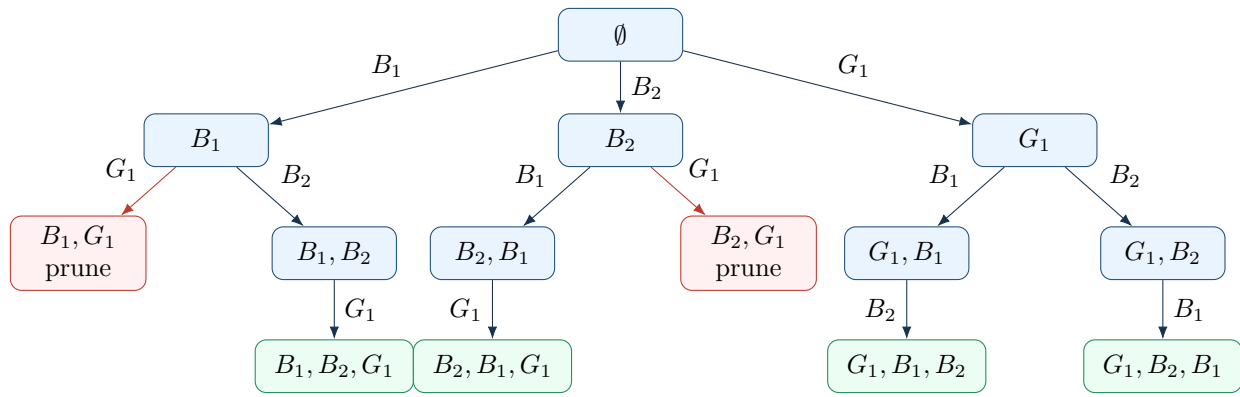
$$B_1, \quad B_2, \quad G_1$$

in three positions, subject to the condition that G_1 must not sit in the middle position.

Total possible arrangements:

$$3! = 3 \cdot 2 \cdot 1 = 6$$

Backtracking prunes a branch as soon as the second position becomes G_1 .



Solution

Forbidden partial state:

second position = G_1 .

Pruned branches:

$(B_1, G_1), \quad (B_2, G_1).$

Valid arrangements:

$G_1, B_1, B_2, \quad G_1, B_2, B_1, \quad B_1, B_2, G_1, \quad B_2, B_1, G_1.$

Answer

Backtracking avoids completing the branches where G_1 is already in the middle.

6 Generic Backtracking Template

Backtracking Template

```

Backtrack(level):
    if a complete solution has been formed:
        process or print the solution
        return

    for each candidate choice:
        make the choice

        if the current partial solution is promising:
            Backtrack(level + 1)

        undo the choice
  
```

7 Correctness Intuition

1. Every possible complete candidate can be reached by some sequence of choices in the state-space tree.

2. The promising function never rejects a partial solution that could still be extended into a valid complete solution.

So every valid solution remains reachable, while impossible branches are skipped.

8 Running Time

If there are at most b choices per level and the solution length is n , the full search tree may contain

$$1 + b + b^2 + \cdots + b^n = \frac{b^{n+1} - 1}{b - 1}$$

nodes.

Worst-case time:

$$\boxed{\mathcal{O}(b^n)}$$

Why Backtracking Still Helps

Pruning can save large subtrees, but many backtracking problems are still exponential in the worst case.

N-Queens Backtracking

1 Problem Definition

N-Queens Problem

Place N queens on an $N \times N$ chessboard so that no two queens attack each other.

No two queens may share the same row, column, or diagonal. The standard backtracking representation places exactly one queen in each row.

Representation

Let

$x[k]$ = column chosen for the queen in row k .

After placing queens in the first k rows, the partial solution is

$$(x[1], x[2], \dots, x[k]).$$

For example, the vector

$$[2, 4, 1, 3]$$

means:

$$Q_1 = (1, 2), \quad Q_2 = (2, 4), \quad Q_3 = (3, 1), \quad Q_4 = (4, 3).$$

2 Mathematical Safety Conditions

Assume queens have already been placed in rows

$$1, 2, \dots, k-1.$$

We want to test whether a new queen can be placed at row k , column c .

2.1 Column Condition

The new queen must not share a column with any earlier queen:

$$x[i] \neq c \quad \text{for every } 1 \leq i < k.$$

2.2 Diagonal Condition

Two squares $(i, x[i])$ and (k, c) lie on the same diagonal exactly when the row difference equals the column difference:

$$|i - k| = |x[i] - c|.$$

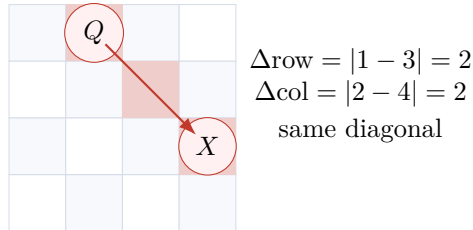
So the new queen is safe from diagonal attack only if

$$|x[i] - c| \neq |i - k| \quad \text{for every } 1 \leq i < k.$$

Complete Safety Test

A queen at row k , column c , is safe if

$$x[i] \neq c \quad \text{and} \quad |x[i] - c| \neq |i - k| \quad \text{for all } 1 \leq i < k.$$



3 Backtracking Search

Each search level corresponds to one row.

Level	Meaning
1	Choose the column for queen Q_1 .
2	Choose the column for queen Q_2 .
$\vdots$	Continue row by row.
N	Choose the column for queen Q_N . A full solution is formed.

Search Rule

Try columns 1 through N in the current row. Recurse only when the chosen column passes the safety test.

N-Queens

```
NQueens(k, n):
    for column = 1 to n:
        if Place(k, column):
            x[k] = column

            if k == n:
                print x[1..n]
            else:
                NQueens(k + 1, n)
```

Safety Test

```
Place(k, column):
    for i = 1 to k - 1:
        if x[i] == column:
            return false

        if abs(x[i] - column) == abs(i - k):
            return false

    return true
```

4 Worked Safety Check

Partial Board

Suppose the partial solution is

$$x[1] = 2, \quad x[2] = 4.$$

Test whether row 3, column 1, is safe.

The new candidate is

$$(k, c) = (3, 1).$$

Check against row 1:

$$x[1] = 2 \neq 1,$$

$$|x[1] - c| = |2 - 1| = 1, \quad |1 - 3| = 2.$$

Since $1 \neq 2$, there is no diagonal conflict with row 1.

Check against row 2:

$$x[2] = 4 \neq 1,$$

$$|x[2] - c| = |4 - 1| = 3, \quad |2 - 3| = 1.$$

Since $3 \neq 1$, there is no diagonal conflict with row 2.

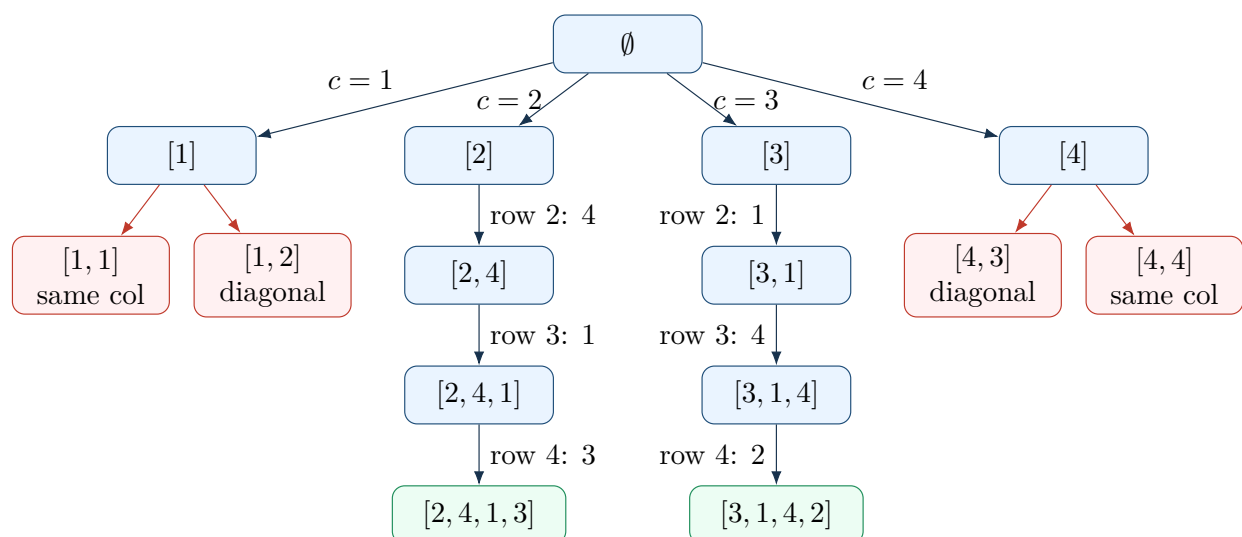
Result

Column 1 is safe for row 3. The partial solution becomes

$$[2, 4, 1].$$

5 State-Space Tree for $N = 4$

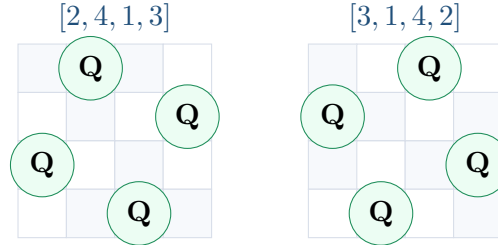
For the 4-Queens problem, many branches are killed by same-column and diagonal conflicts. The diagram shows a useful successful branch and some nearby pruned branches.



6 Two Solutions for $N = 4$

The two standard 4-Queens solutions are

$$\boxed{[2, 4, 1, 3]} \quad \text{and} \quad \boxed{[3, 1, 4, 2]}.$$



Mirror Property

The two 4×4 solutions are mirror images. Reflecting $[2, 4, 1, 3]$ horizontally gives $[3, 1, 4, 2]$.

7 How Backtracking Moves

If a queen is safely placed in row k , the algorithm moves to row $k + 1$. If row $k + 1$ has no safe column, the algorithm returns to row k , removes the old queen, and tries the next column.

Recursive Meaning

$$\text{NQueens}(k, n) = \text{search all safe completions from row } k \text{ to row } n.$$

8 Constructive Pattern: Even-Then-Odd Columns

Backtracking is the general method used in these notes. The source also mentions a direct construction that avoids search for many board sizes.

Main Idea

Place queens row by row using all even-numbered columns first, then all odd-numbered columns.

Even-Then-Odd Order

For a board of size N , define

$$E_N = (2, 4, 6, \dots, 2\lfloor N/2 \rfloor, 1, 3, 5, \dots, 2\lceil N/2 \rceil - 1).$$

Then place the queen in row k at column

$$x[k] = E_N[k].$$

Examples

$$E_4 = (2, 4, 1, 3), \quad E_5 = (2, 4, 1, 3, 5), \quad E_6 = (2, 4, 6, 1, 3, 5).$$

For $N = 6$, this means

$$x[1] = 2, \quad x[2] = 4, \quad x[3] = 6, \quad x[4] = 1, \quad x[5] = 3, \quad x[6] = 5.$$

When the Simple Pattern Works

The simple even-then-odd order gives a valid N-Queens placement when

$$N \bmod 6 \notin \{2, 3\}.$$

Equivalently, it works when the remainder after dividing N by 6 is 0, 1, 4, or 5.

Exceptional Cases

When

$$N \bmod 6 = 2 \quad \text{or} \quad N \bmod 6 = 3,$$

the simple even-then-odd order can create diagonal conflicts. In those cases, the construction must be modified, or the solution can be found using backtracking.

Why Same-Block Conflicts Do Not Occur

First consider two queens inside the even-column block. If their rows are $i < j$, then their row difference is

$$j - i.$$

Their columns are $2i$ and $2j$, so their column difference is

$$|2j - 2i| = 2(j - i).$$

Since

$$2(j - i) \neq j - i,$$

they cannot be on the same diagonal.

The same reasoning applies inside the odd-column block, because odd columns also increase by 2. Therefore:

No diagonal conflict occurs within the even block or within the odd block.

Where Conflicts Can Still Occur

The only possible diagonal conflict is between one queen from the even block and one queen from the odd block.

Let

$$h = \lfloor N/2 \rfloor.$$

An even-column queen in row i has column $2i$. An odd-column queen in row $h + t$ has column $2t - 1$.

These two queens lie on the same diagonal exactly when

$$h + t - i = |2i - (2t - 1)|.$$

Writing

$$a = i - t,$$

the dangerous case becomes

$$h - a = 2a + 1.$$

So

$$3a = h - 1.$$

Thus a conflict occurs exactly when $h - 1$ is divisible by 3. This gives the exceptional congruence classes

$$N \bmod 6 = 2 \quad \text{or} \quad N \bmod 6 = 3.$$

Takeaway

The even-then-odd construction is a useful shortcut, but it is not a complete replacement for backtracking. Use it directly only when

$$N \bmod 6 \notin \{2, 3\}.$$

9 Correctness Intuition

The algorithm is correct because it tries every possible column in each row, but only continues through safe partial boards.

1. If a full vector $[x[1], \dots, x[N]]$ is printed, every pair of queens passed the column and diagonal tests, so the board is valid.
2. If a valid board exists, none of its prefixes violates the safety test. Therefore, backtracking never prunes that valid path.

Conclusion

Backtracking prints exactly the valid N-Queens placements: no invalid placement is printed, and no valid placement is lost by pruning.

10 Complexity

The naive upper bound is N^N , because each of N rows may try N columns. With the column restriction, the search is closer to the permutations of columns:

$$N \cdot (N - 1) \cdot (N - 2) \cdots 1 = N!.$$

Each safety test may compare with up to $N - 1$ previous rows, so a simple implementation has worst-case cost bounded by

$$\mathcal{O}(N \cdot N!)$$

for enumerating the search tree with safety checks.

The recursion depth is N , and the solution vector stores N column values:

$$\text{space} = \mathcal{O}(N)$$

excluding the space used to print or store all solutions.

11 Final Review

N-Queens Essentials

$x[k]$ = column of queen in row k .
 $x[i] \neq x[k]$ prevents column conflicts.
 $|x[i] - x[k]| \neq |i - k|$ prevents diagonal conflicts.
Backtracking searches row by row and prunes unsafe prefixes.

Subset Sum Backtracking

1 Problem Definition

Subset Sum Problem

Given positive integers

$$w[1], w[2], \dots, w[n]$$

and a target value m , find every subset whose sum is exactly m .

A binary solution vector is used:

$$x[i] \in \{0, 1\}.$$

The meaning is

$$x[i] = \begin{cases} 1, & \text{include } w[i], \\ 0, & \text{exclude } w[i]. \end{cases}$$

Solution Equation

A subset is a solution exactly when

$$\sum_{i=1}^n w[i]x[i] = m.$$

2 Example

Use

$$n = 6, \quad m = 30,$$

$$w[1 : 6] = \{5, 10, 12, 13, 15, 18\}.$$

The weights are already sorted in increasing order:

$$5 < 10 < 12 < 13 < 15 < 18.$$

The total sum is

$$5 + 10 + 12 + 13 + 15 + 18 = 73.$$

Root State

The root of the search tree is written as

$$0 \mid 73.$$

The left value is the current selected sum s . The right value is the remaining available sum r .

3 State Representation

At any node, write

$$s \mid r.$$

In these notes, k means the next item to decide. Therefore, before deciding $w[k]$,

$$s = \sum_{i=1}^{k-1} w[i]x[i], \quad r = \sum_{i=k}^n w[i].$$

So r includes the current item $w[k]$ and all later items.

Symbol	Meaning
s	Current sum of the selected elements.
k	Next item index to decide.
r	Sum of items still available from $w[k]$ through $w[n]$.
m	Target sum.
$x[1..n]$	Binary include/exclude solution vector.

At level k , the algorithm has two choices:

$$\boxed{x[k] = 1 \quad \text{include } w[k]} \quad \text{or} \quad \boxed{x[k] = 0 \quad \text{exclude } w[k]}.$$

State Update

If $w[k]$ is included:

$$s_{\text{new}} = s + w[k], \quad r_{\text{new}} = r - w[k].$$

If $w[k]$ is excluded:

$$s_{\text{new}} = s, \quad r_{\text{new}} = r - w[k].$$

4 Bounding Conditions

Backtracking avoids branches that cannot possibly reach the target.

4.1 Overshoot Bound

Since all weights are positive, once the current sum exceeds the target, adding more elements can never fix it:

$$\boxed{s > m \implies \text{prune.}}$$

The next-inclusion bound is

$$\boxed{s + w[k] \leq m.}$$

This says that including the next item $w[k]$ should not already overshoot the target.

4.2 Remaining-Sum Bound

Even if no overshoot occurs, the branch is useless if the remaining values are too small to reach m . If all remaining available values together cannot reach the target, prune:

$$\boxed{s + \sum_{i=k}^n w[i] < m \implies \text{prune.}}$$

Equivalently, a branch may continue only when

$$s + \sum_{i=k}^n w[i] \geq m.$$

Equality Must Be Allowed

The mathematically correct continuation condition is $\geq m$, not $> m$, because equality means the target is still exactly reachable.

5 Promising Function

For a clean mathematical version, suppose $w[1], \dots, w[n]$ are sorted positive integers. At a node before deciding index k , let

$$s = \sum_{i=1}^{k-1} w[i]x[i], \quad R_k = \sum_{i=k}^n w[i].$$

Promising Test

The node is promising when

$$s \leq m \quad \text{and} \quad s + R_k \geq m.$$

The first condition prevents overshoot. The second condition guarantees that enough remaining weight exists to possibly reach the target.

Child Tests

From a promising node before deciding $w[k]$:

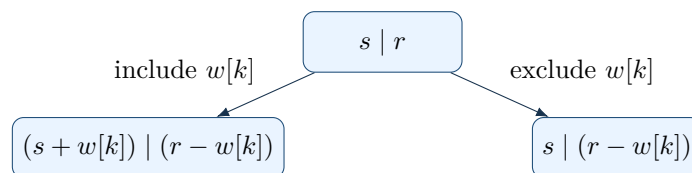
$$\text{include } w[k] \text{ only if } s + w[k] \leq m,$$

and

$$\text{exclude } w[k] \text{ only if } s + (R_k - w[k]) \geq m.$$

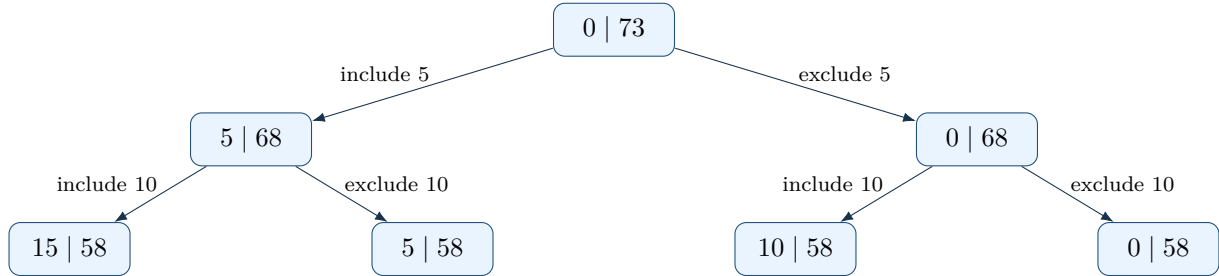
6 Include/Exclude Tree

The state-space tree is binary. Each level decides whether one weight is included or excluded.



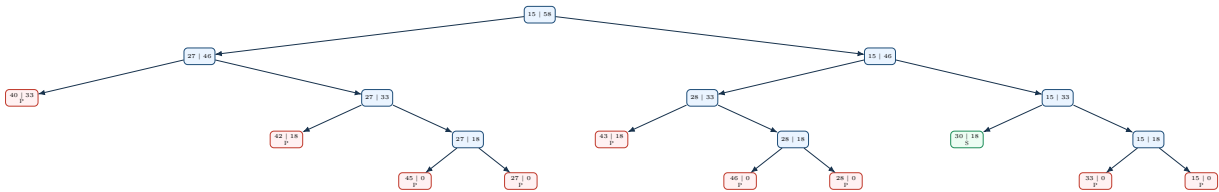
7 Complete Example Tree in Parts

The full tree is too wide for one readable diagram, so it is split into smaller parts. In every subtree, the left child means **include the next item**, and the right child means **exclude the next item**. Green nodes marked S are solutions, and red nodes marked P are pruned.



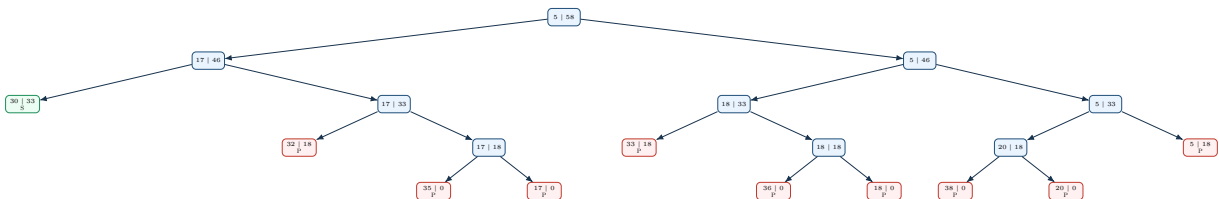
Part A: Include 5, Include 10

This subtree starts from 15 | 58, so the selected sum already contains 5 + 10. The remaining decisions are for 12, 13, 15, 18.



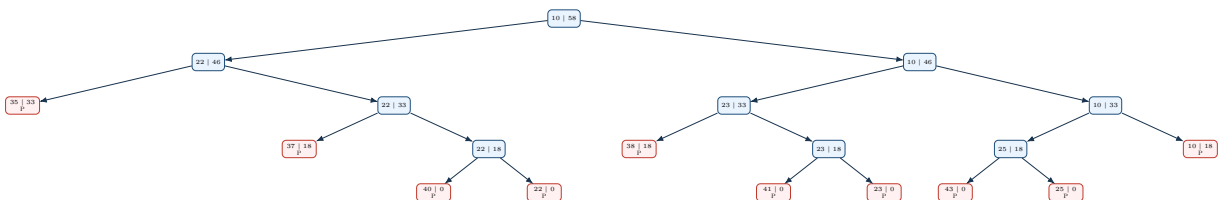
Part B: Include 5, Exclude 10

This subtree starts from 5 | 58. The remaining decisions are for 12, 13, 15, 18.



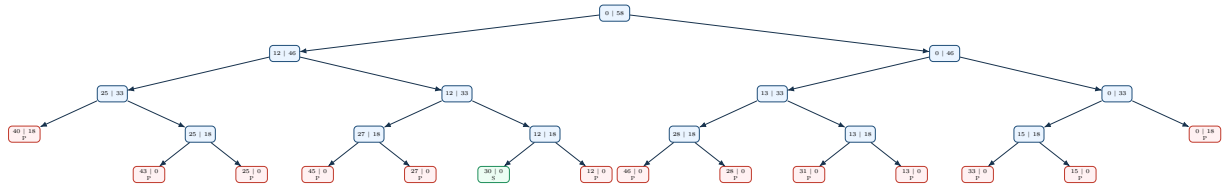
Part C: Exclude 5, Include 10

This subtree starts from 10 | 58. The remaining decisions are for 12, 13, 15, 18.



Part D: Exclude 5, Exclude 10

This subtree starts from $0 \mid 58$. The remaining decisions are for 12, 13, 15, 18.



All Solution Leaves

The complete tree has exactly three solution leaves:

$$\{5, 10, 15\}, \quad \{5, 12, 13\}, \quad \{12, 18\}.$$

8 Checking One Solution Path

Problem

Find a subset of

$$\{5, 10, 12, 13, 15, 18\}$$

whose sum is 30.

Follow the branch

$$5 \rightarrow 10 \rightarrow \text{exclude } 12 \rightarrow \text{exclude } 13 \rightarrow 15.$$

The selected sum is

$$5 + 10 + 15 = 30.$$

Thus the binary vector is

$$x = [1, 1, 0, 0, 1, 0].$$

Check the solution equation:

$$\begin{aligned} \sum_{i=1}^6 w[i]x[i] &= 5(1) + 10(1) + 12(0) + 13(0) + 15(1) + 18(0). \\ &= 5 + 10 + 0 + 0 + 15 + 0 = 30. \end{aligned}$$

Checked Path

This branch gives the valid subset

$$\{5, 10, 15\}.$$

The other solution leaves are already listed at the end of the complete tree.

9 Pseudocode

The pseudocode uses

$$\text{SubsetSum}(k, s, r),$$

where k is the next item to decide, s is the current selected sum, and r is the remaining sum from $w[k]$ onward. Initially,

$$\text{SubsetSum}(1, 0, \sum_{i=1}^n w[i]).$$

Subset Sum Backtracking

```
SubsetSum(k, s, r):
    if s == m:
        print the selected set
        return

    if k > n:
        return

    if s > m or s + r < m:
        return

    x[k] = 1
    if s + w[k] <= m:
        SubsetSum(k + 1, s + w[k], r - w[k])

    x[k] = 0
    if s + r - w[k] >= m:
        SubsetSum(k + 1, s, r - w[k])
```

Why This Version Is Cleaner

Because k is the next item to decide, the algorithm tests $w[k]$, not $w[k + 1]$. The boundary case is handled directly by

$$k > n.$$

10 Correctness Intuition

The include/exclude tree contains every possible subset, because each element receives exactly one binary decision:

$$x[i] = 1 \quad \text{or} \quad x[i] = 0.$$

The algorithm prints only when the selected sum equals m , so every printed subset is valid. The bounds prune only when the sum is already too large or the remaining values are insufficient, so no reachable solution is discarded.

Conclusion

Backtracking searches the subset tree, but the bounds remove branches that cannot satisfy

$$\sum_{i=1}^n w[i]x[i] = m.$$

11 Complexity

Without pruning, there are two choices for each of n elements:

$$2^n$$

possible binary vectors.

Therefore, the worst-case time remains exponential:

$$\mathcal{O}(2^n).$$

The recursion depth is at most n , and the solution vector has length n , so the auxiliary space is

$$\mathcal{O}(n).$$

Practical Effect

Sorting and bounding do not remove the exponential worst case, but they often reduce the number of recursive calls dramatically.

12 Final Review

Subset Sum Essentials

$x[i] \in \{0, 1\}$ records exclude/include decisions.
 $\sum_{i=1}^n w[i]x[i] = m$ defines a solution.
 $s > m$ means overshoot, so prune.
 $s + R_k < m$ means insufficient remaining sum, so prune.
After one solution is printed, continue searching for others.

M-Colouring Backtracking

1 Problem Definition

M-Colouring Problem

Given a graph

$$G = (V, E)$$

and M available colours, assign one colour to each vertex so that no two adjacent vertices receive the same colour.

Let

$$V = \{1, 2, \dots, N\}, \quad C = \{1, 2, \dots, M\}.$$

The solution array is

$$x[v] = \text{colour assigned to vertex } v.$$

Colouring Constraint

For every edge $(u, v) \in E$,

$$x[u] \neq x[v].$$

If the colours are written as

$$\{R, G, B\},$$

then $M = 3$, and each $x[v]$ must be one of R , G , or B .

2 Three Versions

Version	Question
Decision	Can G be coloured using at most M colours?
Optimisation	What is the minimum number of colours needed?
Counting	How many valid M -colourings exist?

Chromatic Number

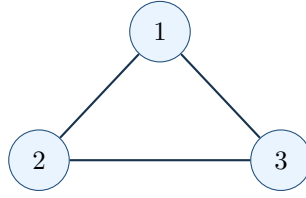
The minimum number of colours required to colour G is called the chromatic number and is denoted by

$$\chi(G).$$

3 Simple No-Solution Example

A triangle graph has three vertices and every pair of vertices is adjacent:

$$E = \{(1, 2), (2, 3), (1, 3)\}.$$



Try to colour this triangle with $M = 2$ colours, say R and G .

$$x[1] = R.$$

Since vertex 2 is adjacent to vertex 1, it must use the other colour:

$$x[2] = G.$$

Vertex 3 is adjacent to both 1 and 2, so it cannot use R and cannot use G . Therefore,

no valid 2-colouring exists.

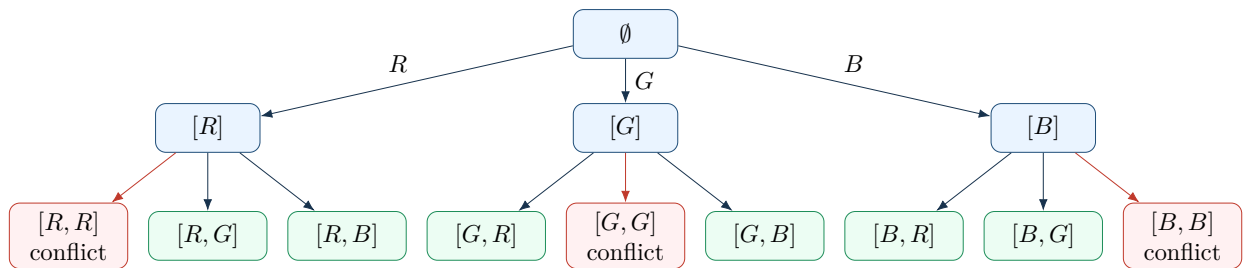
4 Backtracking State

At level k , the algorithm chooses a colour for vertex k . The partial solution is

$$(x[1], x[2], \dots, x[k]).$$

The state-space tree has at most M branches per level:

branching factor = M .



Example shown for two adjacent vertices: equal colours are pruned immediately.

5 Safety Test

A colour c is safe for vertex v if no already coloured neighbour of v has colour c .

Safety Condition

Let

$$N(v) = \{u \in V : (u, v) \in E\}$$

be the neighbour set of v . Colour c is safe for v when

$$x[u] \neq c \quad \text{for every already coloured } u \in N(v).$$

Safe

```
Safe(v, c):  
    for every vertex u adjacent to v:  
        if colour[u] == c:  
            return false  
  
    return true
```

6 Graph Colouring Algorithm

All M-Colourings

```
GraphColouring(vertex):  
    if vertex > numberOfVertices:  
        print colour[1..numberOfVertices]  
        return  
  
    for c = 1 to M:  
        if Safe(vertex, c):  
            colour[vertex] = c  
  
            GraphColouring(vertex + 1)  
  
            colour[vertex] = 0
```

The assignment

$$\text{colour}[\text{vertex}] = 0$$

removes the colour during backtracking so that the next branch starts from the correct partial colouring.

7 Worked Example with Search Tree

Problem

Colour the path graph

$$1 - 2 - 3 - 4$$

using $M = 3$ colours R, G, B .

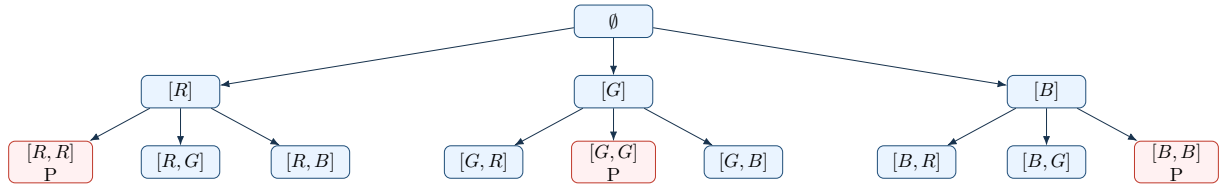


The algorithm colours vertices in the order 1, 2, 3, 4, trying colours in the order R, G, B .

Tree Notation

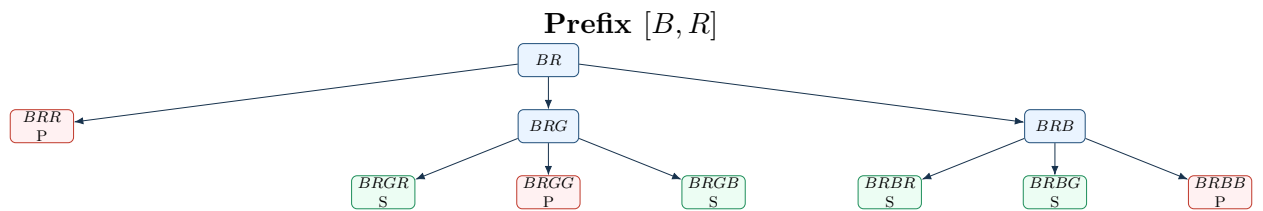
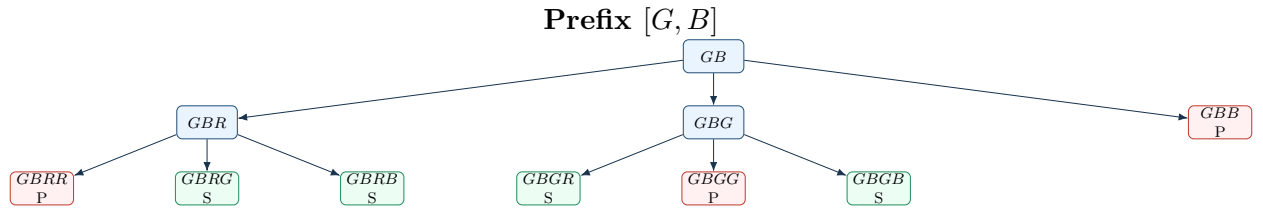
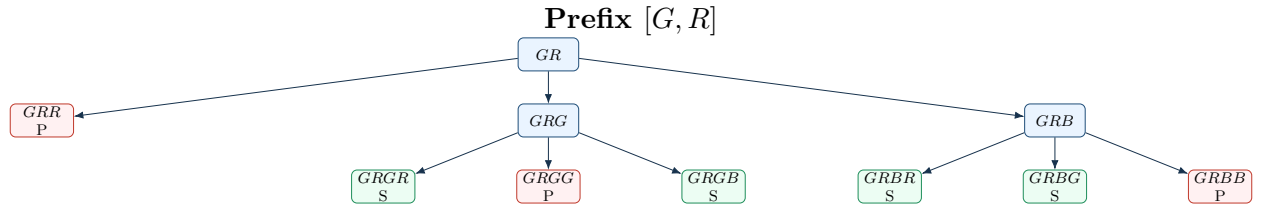
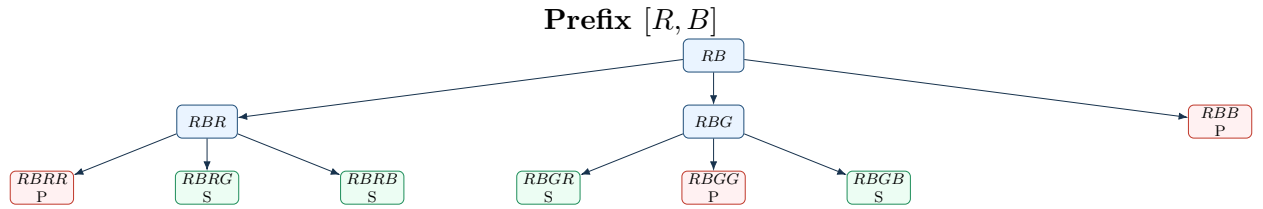
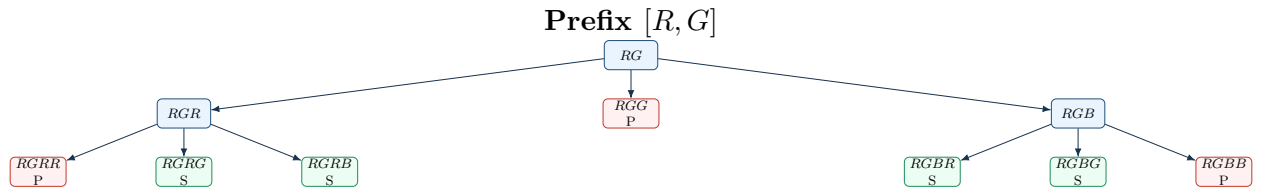
A red node marked P is pruned because it gives the same colour to two adjacent vertices. A green node marked S is a complete valid colouring.

First Two Vertices

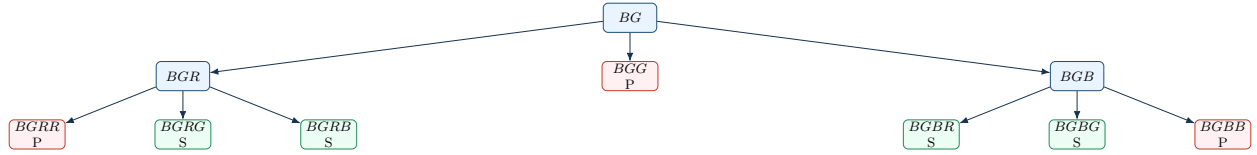


Remaining Subtrees

Each subtree below continues from one valid prefix after colouring vertices 1 and 2. The next two levels choose colours for vertices 3 and 4. To keep the tree readable, compact labels such as $RGRB$ mean the colour vector $[R, G, R, B]$.



Prefix $[B, G]$



Worked Example Result

The path graph has exactly

$$3 \cdot 2 \cdot 2 \cdot 2 = 24$$

valid 3-colourings. For example, $[R, G, R, G]$ and $[B, R, G, B]$ are valid complete paths.

8 First Solution vs All Solutions

Variant	Behaviour
First solution	Return immediately after one complete valid colouring is found.
All solutions	Print a complete colouring, backtrack, and continue searching.

9 Correctness Intuition

Every complete colouring corresponds to one path in the state-space tree because each vertex receives exactly one colour. The algorithm tries every colour for every vertex, but it recurses only when the partial colouring is safe.

1. If the algorithm prints a colouring, every edge has passed the safety test, so adjacent vertices have different colours.
2. If a valid colouring exists, none of its prefixes violates the safety test, so backtracking never prunes that valid path.

Conclusion

The algorithm prints exactly the valid M -colourings of the graph.

10 Complexity

In the worst case, each of the N vertices can try M colours:

$$M \cdot M \cdots M = M^N.$$

So the worst-case running time is

$$\mathcal{O}(M^N).$$

The recursion depth is N , and the colour array stores one value for each vertex:

$$\text{auxiliary space} = \mathcal{O}(N).$$

11 Final Review

M-Colouring Essentials

$G = (V, E), \quad |V| = N, \quad C = \{1, \dots, M\}.$

$x[v]$ = colour assigned to vertex v .

$(u, v) \in E \Rightarrow x[u] \neq x[v].$

Worst-case time = $\mathcal{O}(M^N)$, space = $\mathcal{O}(N)$.

Hamiltonian Cycle Backtracking

1 Problem Definition

Hamiltonian Cycle

A **Hamiltonian cycle** in a graph is a cycle that visits every vertex exactly once and returns to the starting vertex.

For a graph with n vertices, a Hamiltonian cycle has the form

$$v_1, v_2, \dots, v_n, v_1.$$

The first n vertices are all distinct:

$$v_i \neq v_j \quad \text{whenever } i \neq j,$$

and every consecutive pair must be connected by an edge:

$$(v_i, v_{i+1}) \in E \quad \text{for } 1 \leq i < n, \quad (v_n, v_1) \in E.$$

Cycle Conditions

visit every vertex exactly once,
use only existing graph edges,
return from the last vertex to the first vertex.

2 Representation

Use an adjacency matrix

$$G[1..n][1..n],$$

where

$$G[i][j] = \begin{cases} 1, & \text{if vertices } i \text{ and } j \text{ are adjacent,} \\ 0, & \text{otherwise.} \end{cases}$$

Use a solution array

$$x[1..n],$$

where $x[k]$ is the vertex placed at position k in the cycle.

Fixing the First Vertex

Set

$$x[1] = 1.$$

This avoids generating the same cycle repeatedly by rotation.

The remaining positions are initialized as

$$x[2] = x[3] = \dots = x[n] = 0.$$

The search begins from position

$$\text{Hamiltonian}(2).$$

3 Valid Next Vertex

At position k , a candidate vertex v may be assigned to $x[k]$ only if it passes the required tests.

3.1 Adjacency to Previous Vertex

The new vertex must be adjacent to the previous vertex:

$$G[x[k-1]][v] \neq 0.$$

3.2 No Repeated Vertex

The new vertex must not already appear in the partial path:

$$v \notin \{x[1], x[2], \dots, x[k-1]\}.$$

3.3 Closing Edge at the End

If $k = n$, the last selected vertex must connect back to the first vertex:

$$G[v][x[1]] \neq 0.$$

Safety Test

A candidate v is valid at position k when

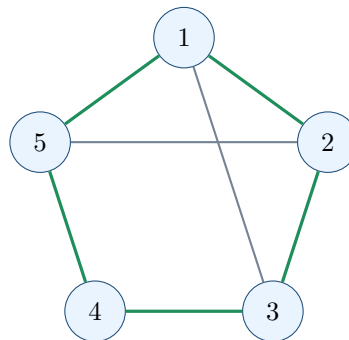
$$G[x[k-1]][v] \neq 0 \quad \text{and} \quad v \notin \{x[1], \dots, x[k-1]\}$$

and, for $k = n$, also

$$G[v][x[1]] \neq 0.$$

4 Example Graph

Consider the graph below.



One Hamiltonian cycle is

$$1, 2, 3, 4, 5, 1.$$

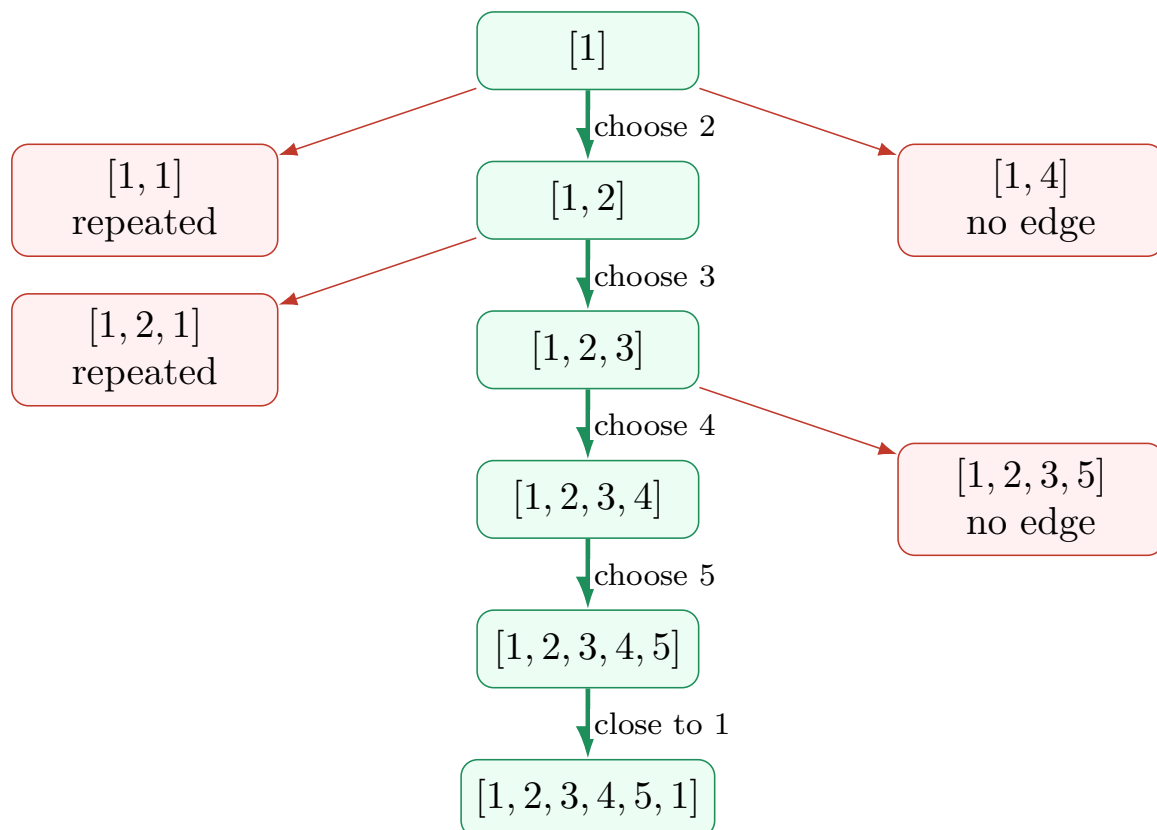
For this graph, the adjacency matrix is

$$G = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix}.$$

5 Backtracking Search

At position k , the algorithm tries candidate vertices one by one. A branch is pruned when:

- the candidate is not adjacent to the previous vertex,
- the candidate has already been used,
- the last candidate cannot connect back to $x[1]$.



Search Meaning

The green branch forms a valid Hamiltonian cycle. The red branches fail because they repeat a vertex or require a missing edge.

6 Algorithm

Main Idea of the Algorithm

The algorithm builds the cycle one position at a time in the array

$$x[1], x[2], \dots, x[n].$$

Here $x[k]$ means the vertex placed at position k of the cycle. After the array is filled, the printed cycle is

$$x[1], x[2], \dots, x[n], x[1].$$

The last $x[1]$ is not stored again in the array; it is checked using the final closing-edge test.

Why the First Vertex Is Fixed

The algorithm sets

$$x[1] = 1$$

because a cycle can be rotated. For example,

$$1, 2, 3, 4, 5, 1$$

and

$$2, 3, 4, 5, 1, 2$$

describe the same cycle with a different starting point. Fixing $x[1] = 1$ avoids generating the same cycle repeatedly by rotation.

Initialization

```
x[1] = 1
x[2..n] = 0
Hamiltonian(2)
```

Hamiltonian Search

```
Hamiltonian(k):
    while true:
        NextValue(k)

        if x[k] == 0:
            return

        if k == n:
            print x[1..n], x[1]
        else:
            Hamiltonian(k + 1)
```

Meaning of $\text{Hamiltonian}(k)$

$\text{Hamiltonian}(k)$ means: choose a valid vertex for position k . The positions $x[1], x[2], \dots, x[k-1]$ already form a valid partial path. The function tries to extend this path by assigning a suitable value to $x[k]$.

How the Recursive Search Works

After $\text{NextValue}(k)$ returns, there are two possibilities:

- If $x[k] = 0$, no valid candidate remains for position k , so the algorithm returns and backtracks.
- If $x[k] \neq 0$, a valid candidate was found. If $k = n$, the cycle is complete and is printed. Otherwise, the algorithm recursively fills position $k + 1$.

NextValue

```
NextValue(k):
    while true:
        x[k] = (x[k] + 1) mod (n + 1)

        if x[k] == 0:
            return

        if G[x[k - 1]][x[k]] != 0:
            for j = 1 to k - 1:
                if x[j] == x[k]:
                    break

            if j == k:
                if k < n:
                    return

            if k == n and G[x[n]][x[1]] != 0:
                return
```

Candidate Tests Performed by $\text{NextValue}(k)$

A candidate $x[k]$ is accepted only when it satisfies the required tests:

$$G[x[k - 1]][x[k]] \neq 0$$

checks that the new vertex is adjacent to the previous vertex,

$$x[k] \notin \{x[1], x[2], \dots, x[k - 1]\}$$

checks that the vertex has not already been used, and if $k = n$,

$$G[x[n]][x[1]] \neq 0$$

checks that the last vertex connects back to the starting vertex.

Meaning of 0

The value $x[k] = 0$ means that no more candidate vertices are available at position k . The update

$$x[k] = (x[k] + 1) \bmod (n + 1)$$

tries candidates in the order

$$1, 2, 3, \dots, n.$$

After all candidates have been tried, the value becomes 0. This tells the algorithm to backtrack to the previous position.

Backtracking Interpretation

The algorithm grows a partial path only while it remains valid. As soon as a candidate repeats a vertex, misses an edge, or fails the closing-edge test, that branch is abandoned. Then the algorithm goes back to the previous position and tries the next candidate.

7 Worked Trace

For the example graph, start with

$$x[1] = 1.$$

Choose 2:

$$G[1][2] = 1, \quad 2 \notin \{1\},$$

so

$$x[2] = 2.$$

Choose 3:

$$G[2][3] = 1, \quad 3 \notin \{1, 2\},$$

so

$$x[3] = 3.$$

Choose 4:

$$G[3][4] = 1, \quad 4 \notin \{1, 2, 3\},$$

so

$$x[4] = 4.$$

Choose 5:

$$G[4][5] = 1, \quad 5 \notin \{1, 2, 3, 4\}.$$

Since this is the last position, also check the closing edge:

$$G[5][1] = 1.$$

Answer

Therefore,

$$\boxed{1, 2, 3, 4, 5, 1}$$

is a Hamiltonian cycle.

8 Correctness Intuition

The search tree considers possible vertex permutations after fixing $x[1] = 1$. The adjacency test ensures every consecutive pair is an edge. The repetition test ensures every vertex appears at most once. The final closing-edge test ensures the path is a cycle.

1. Any printed sequence uses valid edges, has no repeated internal vertex, and returns to the start.
2. Any Hamiltonian cycle starting at vertex 1 corresponds to one branch of the search tree, so it will be found unless pruned by an invalid condition.

9 Complexity

After fixing the first vertex, the algorithm searches through permutations of the remaining vertices:

$$(n - 1)!.$$

Thus the common factorial bound is

$$\boxed{\mathcal{O}(n!)}.$$

A looser upper bound is obtained by saying that each of n positions may try up to n vertices:

$$\boxed{\mathcal{O}(n^n)}.$$

The factorial bound is sharper because the algorithm rejects repeated vertices.

The solution array and recursion depth use linear auxiliary space:

$$\boxed{\mathcal{O}(n)}.$$

10 Final Review

Hamiltonian Cycle Essentials

$x[1] = 1$ fixes the starting vertex.
 $G[x[k - 1]][x[k]] \neq 0$ checks adjacency.
 $x[k] \notin \{x[1], \dots, x[k - 1]\}$ prevents repetition.
 $G[x[n]][x[1]] \neq 0$ closes the cycle.
Time = $\mathcal{O}(n!)$, space = $\mathcal{O}(n)$.